ECE 554

FINAL PROJECT PRESENTATION

AUTONOMOUS PARKING SYSTEM FOR AUTONOMOUS VEHICLES

Presented by:

Shaurya Doger(73212206),Kassandra Rodriguez(61042524),Bhuvaneshwar(76886265)

TABLE OF CONTENTs

- Introduction
- Requirements Analysis
- System Design
- Implementation
- Testing
- Deployment
- Maintenance
- Conclusion
- References
- Future Work

INTRODUCTION

Today, semi-autonomous vehicles are becoming more and more popular. Almost every car company is developing their own AV. One of the known challenges for AV's is automated parking. The vehicle is expected to park on its own without crashing into anything or injuring anyone around it. Most car companies have developed assistive parking technology, but most systems require the driver to provide some assistance in case the vehicle is unable to perform the action. While previous works have been performed by researchers and engineers, studies of [1], [2], and [3] have indicated several weaknesses including limited sensor variety, incomplete robot hardware implementation, limited hardware interfacing mapping algorithms, insufficient obstacle detection prediction, etc. We want to develop our own autonomous parking system to help fix/improve the issues emphasized above.

Objectives

- **Develop a autonomous parking system**
  - Design and and build a system that can maneuver through a controlled environment and autonomously park itself in the next available spot.
- **Use sensors for localization**
  - Utilize a camera module and an ultrasonic sensor to allow the robot to detect obstacles such as parked vehicles. The sensors should help the robot navigate to the next available parking spot.
- **Build a functional robot**
  - Build a robot that can steer and drive forwards and backwards. The robot should also have the space to mount servo motors that can be used to move the camera module to detect obstacles.
- **Test and validate**
  - Each component of the robot should be individually tested to confirm that everything can work together. Example: Camera, steering, ultrasonic sensor readings
- **Document results:**
  - All code should be properly documented as well as documentation of the system design and the testing results.

Requirement Analysis

Functional Requirements:

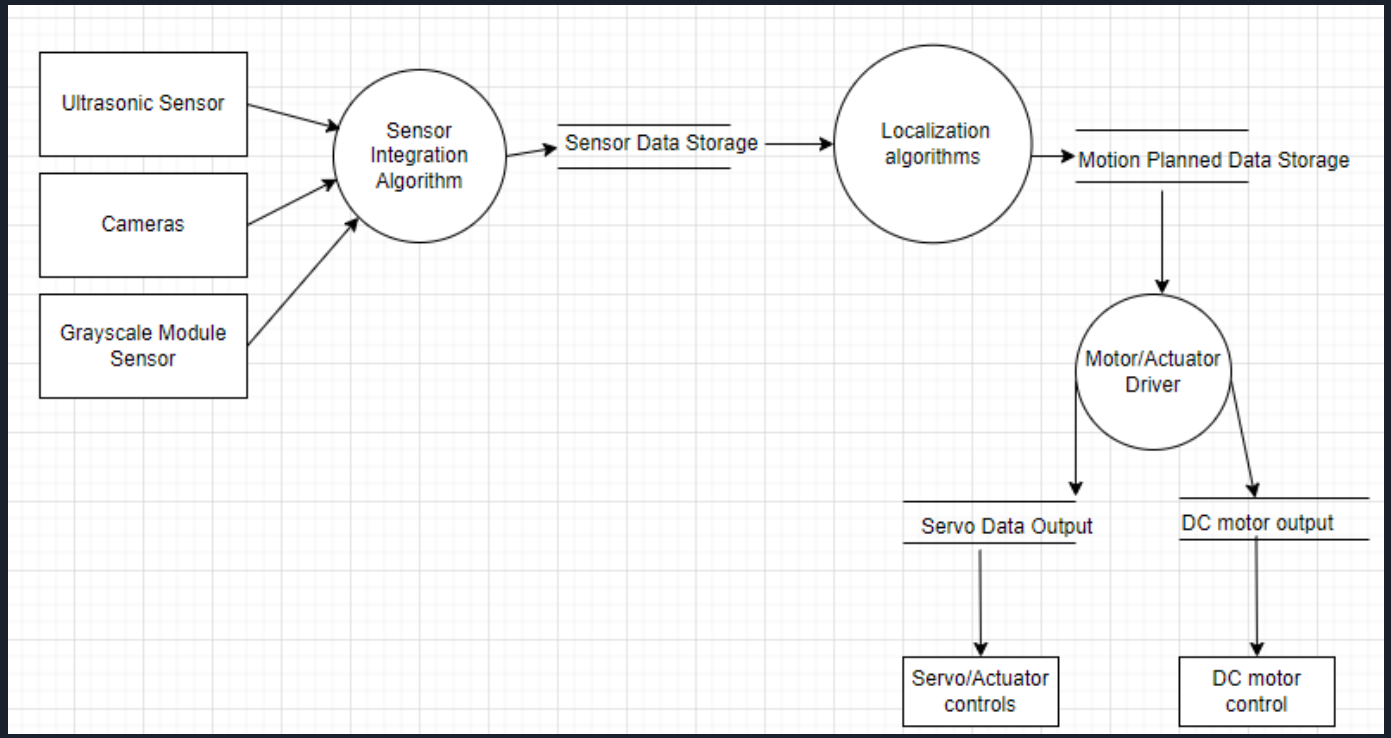
- **Sensors**
  - the use of Lidar/ultrasonic sensors and camera to detect obstacles
- **Localization**
  - Determine the robot's position on the controlled environment using it's sensors
- **Control**
  - Control the robots braking, acceleration, etc.
- **User interface**
  - Create a web-based app or simple mobile application to track the data from the robot.

Requirement Analysis

Non-functional Requirements:

- **User interface:**
  - Make sure the user interface we decide to use is user friendly. Something as easy as running the main program from the command line.
- **Reliability:**
  - Make sure the system works accurately in the expected environment. The robot must be able to get to an available parking spot without encountering any issues.
- **Braking System & Reset Actuators:**
  - Implement a braking system that will stop the robot and reset the actuators so that they are ready to be used the next time we run the system.

I/O Design


Ultrasonic Sensor
Cameras
Grayscale Module Sensor
Sensor Integration Algorithm
Sensor Data Storage
Localization algorithms
Motion Planned Data Storage
Motor/Actuator Driver
Servo Data Output
DC motor output
Servo/Actuator controls
DC motor control

System Design: High-Level Design

Architecture

- **Perception Layer**
  - This layer consists of the camera and ultrasonic sensor being used as the main data collecting components.
- **Decision Layer**
  - This layer consists of the control logic that is used to processes the data to make the proper parking decision. Example: “Spot is empty” and “Spot is not empty.”
- **Execution layer**
  - In this layer the actuators play a major role since they are the ones responsible for the robot’s movement. Example: Moving forward/backward, steering, and panning/tilting the camera module.

System Design: Low-Level Design

Sensor Integration

- **Camera:**
  - The camera module is used to provide visual confirmation that the potential parking spot is actually available.
- **Ultrasonic sensor:**
  - The ultrasonic sensor is used to measure the distance between any obstacles as the robot is driving forward and inspecting the parking lot.
- **Localization:**
  - The use of both the camera and ultrasonic sensor allowed us to simplify the localization process by relying on relative positioning from these sensors during the main parking process.

System Design: Low-Level Design

- **Microcontroller**
  - The raspberry Pi utilizes a motor controller called a *Robot Hat*. The *robot hat* works closely with a python library called *Picarx*. The *Picarx* library is essential since it's used to execute all of the control logic.
- **Steering & Movement:**
  - All of the actuators on the robot are controlled by the microcontroller using the following function calls: *forward, backward, stop, set_dir_servo_angle, set_cam_pan_angle and set_cam_tilt_angle.*

Control Algorithms

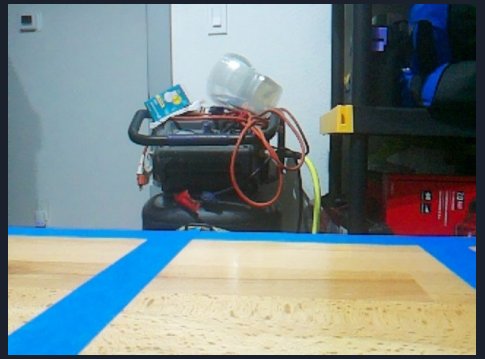
- **Image Processing:**
  - We utilize image processing to identify if the potential available spot has enough of a specific color to be considered available. If not much of that specific color is found, then the spot is not available for parking.
- **Decision Making:**
  - This system uses a simple decision making process that uses the ultrasonic sensor distance data and the camera's visual information.
- **Feedback Control:**
  - If the ultrasonic sensor thinks it finds an available spot and the camera processes the information and determines it's not available, feedback is sent back to the microcontroller and the robot is instructed to continue searching for the next available spot.
- **Safety/Error Handling:**
  - Within the main functions, we included various amounts of error handling scenarios. Some of those scenarios include, the ability to interrupt the run manually and safely stop the robot.

Implementation: Hardware

- **Microcontroller & Expansion Board:**
  - For this project we used a Raspberry Pi 4 and a Robot Hat expansion board that is used to utilize all of our actuators (2 DC motors, 3 servo motors) and sensors (ultrasonic sensor and camera module).
- **Robot Platform:**
  - We used the sunfounder Picar kit as the base of our self parking system.
  - This robot kit includes a metal chassis that is durable and capable of going through a large number of testing scenarios.
  - The kit also included the camera module and the ultrasonic sensor used in our system.
- **Sensor Setup:**
  - When adapting the robot to meet our system requirements, we decided to place the camera module facing forward and the ultrasonic sensor was mounted on the side of the camera module mount.
  - The purpose of this placement was to keep the ultrasonic sensor facing to the right of the robot while its scanning the area and collecting distance data as the robot drives forward/backward.
- **Power Supply:**
  - We started using two 3.4 V rechargeable batteries but the rechargeable attachment stopped working so we purchases a 7.2 V battery pack and its charger.
  - The Robot Hat can take **7 V to 12V** as an input so this power supply worked perfectly.

Implementation: Software

- **Camera Configuration:**
  - We used the Vilib python library to set up the camera and it also allows us to view real time footage as the system is running.
- **Ultrasonic Sensor Data:**
  - We utilize the Picarx library to use the ultrasonic sensor.
  - The ultrasonic sensor checks for a specific threshold as the robot is moving forward.
  - If the threshold is greater than 20, then the sensor tells the robot that it has detected an available spot.



```
def find_parking_spot(px, distance_threshold):
    """ Here we use the ultrasonic sensor to continuously search for a threshold greater that 20
        Drive forward = px.backward
        Drive backward = px.forward
    """

    # We move backward because for some reason backward is actually forward      You, 17 hours ago
    px.backward(speed=1)
    try:
        while True:
            distance = px.ultrasonic.read()
            if distance > distance_threshold:
                print("Potential empty spot detected.")
                px.backward(0)  # Stop the car
                confirm_parking_spot(px)
                break
            time.sleep(0.5)
    except KeyboardInterrupt:
        print("Manual interruption.")
    finally:
        px.stop()
        print("Finished scanning for parking spots.")
```

Implementation: Software

Image Processing:

- The image processing work mainly happens in the is_parking_spot_empty() function. The camera takes a picture of the area that needs to be examined.
- Then the function reads the image and converts it from BGR (Blue, Green, Red) format to HSV (Hue, Saturation, Value) format.
- We do this conversion because we found it as an efficient way to extract a specific color from the image.
- Then we use two numpy arrays to define the lower and upper bounds of the specific color we wish to detect. In this case we are looking for blue. We set our values to detect for dark/bright shades of blue.
- Then we create a mask where the sections of blue we preferred are isolated as white pixels. Then we clean up the newly created mask to get rid of noise after the preprocessing.
- Then we calculate the coverage ratio by getting the total number of white pixels and dividing it by the total number of pixels.
- If the coverage_ratio is greater than 5% then there is a sufficient amount of blue to determine that the spot is in fact available to park in.

```python
def is_parking_spot_empty(image_path):
    """
        Here we check if the parking spot is empty. We convert the still frame into a HSV format
        so that we can better identify blue spots in the image
    """
    image = cv2.imread(image_path)
    if image is None:
        print("Error: Image not found or cannot be loaded.")
        return False

    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # We find that these value are close to the tape color we are using in our
    # controlled env
    lower_blue = np.array([100, 150, 50])
    upper_blue = np.array([140, 255, 255])
    mask = cv2.inRange(hsv, lower_blue, upper_blue)

    # Creates a small square array used for image processing
    kernel = np.ones((5, 5), np.uint8)

    # helps remove small white noise
    mask_cleaned = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

    # calculates the proportion of the mask that is blue
    coverage_ratio = np.sum(mask_cleaned > 0) / mask_cleaned.size
```

Implementation: Software

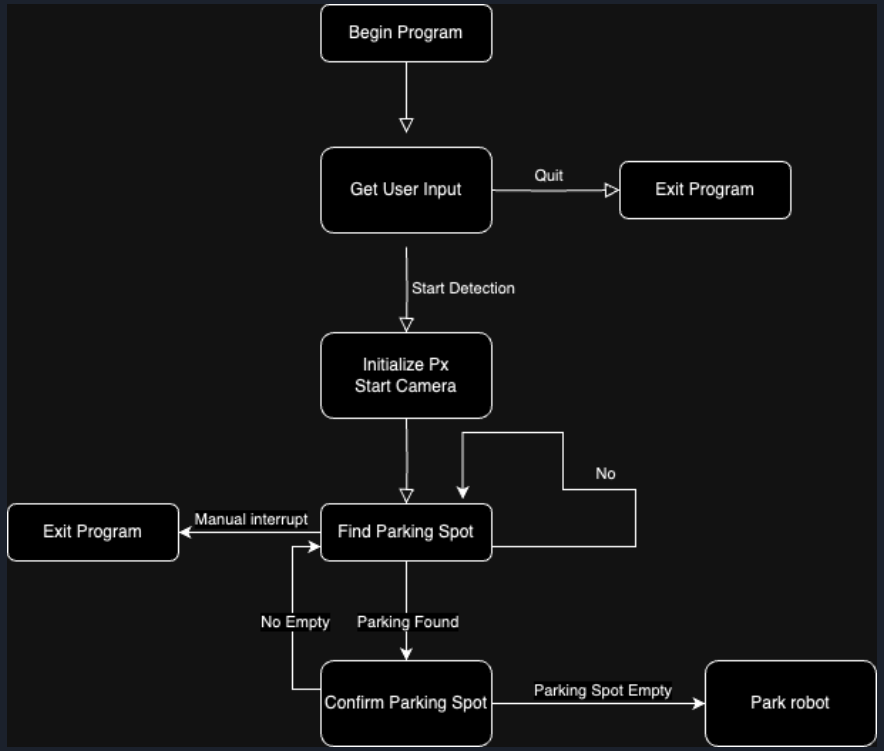
Control Logic

Movement:

- Most of our robot movements to maneuver into the parking spot is found in the confirm_parking_spot() function.
- If the camera confirms that the spot is empty, we initiate the process to park.
- We first back up a little bit to account for our turn radius.
- We then steer the right and start driving forward to maneuver into the parking space.
- In the end, we rest our steering servo motor so that it goes back to its initial state.

```python
def confirm_parking_spot(px):
    right_angle = 90
    down_angle = -20  # Tilt the camera down to focus on the ground
    px.set_cam_pan_angle(right_angle)
    px.set_cam_tilt_angle(down_angle)
    time.sleep(1)  # Wait for the camera to stabilize

    print("Camera adjusted, capturing image for confirmation.")
    _time = time.strftime("%y-%m-%d_%H-%M-%S", time.localtime())
    path = "/home/kassandrarodriguez/auto-park-car/photos/"
    Vilib.take_photo(str(_time), path)
    image_path = f"{path}/{_time}.jpg"
    print(f"The photo saved as: {image_path}")

    if is_parking_spot_empty(image_path):
        print("The parking spot is empty.")
        # Drive into the parking spot
        px.forward(speed=2)
        time.sleep(0.4)

        # Steer right
        px.set_dir_servo_angle(25)
        time.sleep(0.5)
        px.backward(speed=1)
            You, 3 minutes ago • Uncommitted changes
        # Allows us time to drive into the parking spot
        time.sleep(4)
        px.stop()

        # Reset steering angle
        px.set_dir_servo_angle(0)
    else:
        print("The parking spot is not empty.")
```

Drive.py Flow Chart



Testing

The testing phase of this project was very important because it helps us get all of the parts of this project working before we bring them all together. This ensures that our system is reliable and that it meets all requirements.

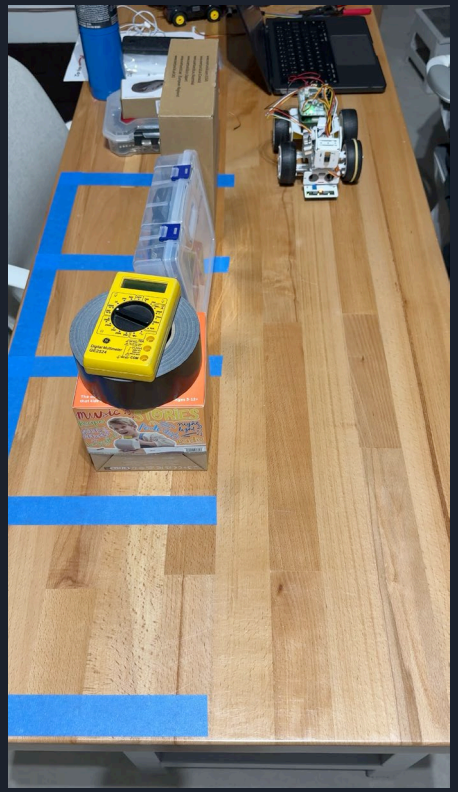
We created individual python scripts to test every part of our robot:

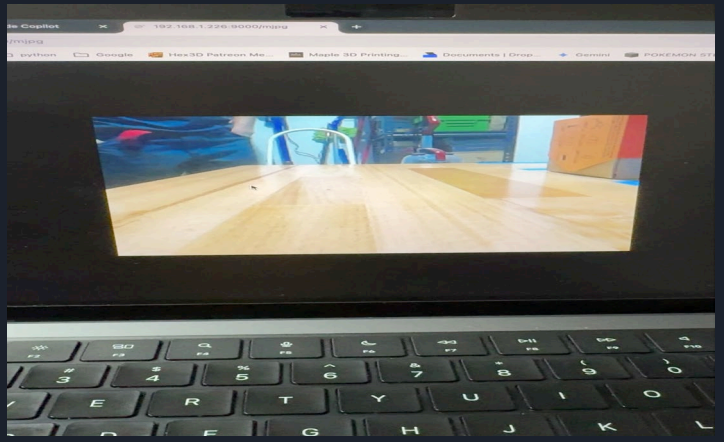
- **Test_camera_movement.py:**
  - Used to test that we can view the real time data from the camera. It also tests that the servo motors work properly by panning and tilting the motors that are holding the camera in place.
- **Test_image_viewer.py:**
  - This test script checks that we can take a picture/frame and save it onto the raspberry pi. The function actually saves it to a location on the repo so we can commit those changes from the raspberry pi and pull them from our development laptop.
- **Test_ultrasonic_sensor.py**:
  - this script tests that the ultrasonic sensor is able to detect obstacles in front of it. We also used the script to come up with a good value to use as our threshold in the future.
- **Test_drive_turn.py**:
  - this script was created to test the forward driving movement along with the ultrasonic data collection. If we find a spot greater than 20, the camera turns and this tells us we found a spot.

Deployment

- Hardware Setup:
  - The final set up for our robot involved us moving the ultrasonic sensor to the right side of the robot. Initially it was set to be in the front facing the same direction as the camera module but placement didn't satisfy out system requirements.
  - The power supply was replaced as well. We mounted the new power supply using zip ties and allowed it enough room for the wire to connect to the Robot Hat and power the Raspberry Pi and also have enough room to charge the battery.
- Software Configuration:
  - All users needed to install the required libraries to run the main script for the auto parking system.
  - Install documentation was included in the read me file
- Launch:
  - The final script was used in a controlled environment.
  - We put together a parking lot set up using blue masking tape and used boxes to resemble the vehicles being parked in the parking lot.

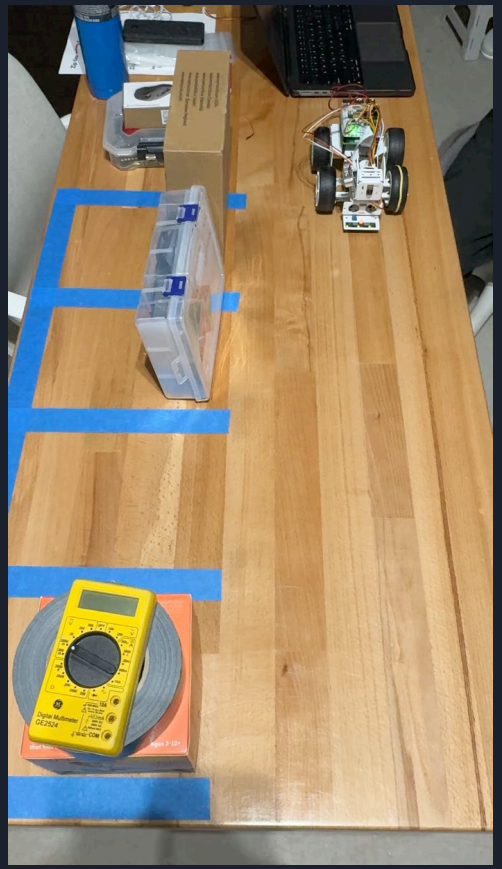
Demo 1

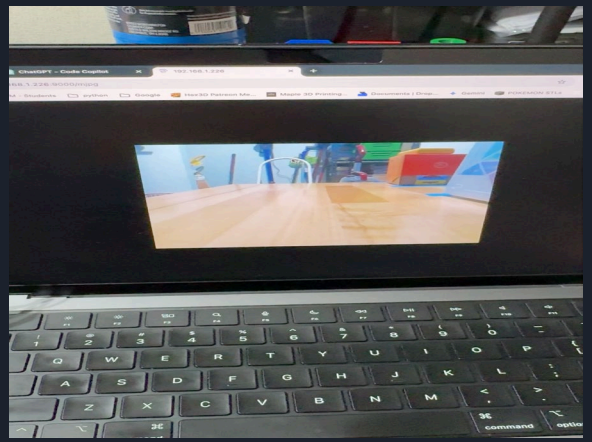




```
[kassandrarodriguez@raspberrypi:~/auto-park-car $ python3 drive.py
vilib 0.2.1 launching ...
picamera2 0.3.17
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
local display failed, because there is no gui.
Press ENTER to start the parking detection or 'Q' to quit.
Input: Starting network video streaming ...

Running on: http://192.168.1.226:9000/mjpg

 * Serving Flask app 'vilib.vilib'
 * Debug mode: off

Starting parking detection...
Potential empty spot detected.
Camera adjusted, capturing image for confirmation.
The photo saved as: /home/kassandrarodriguez/auto-park-car/photos//24-04-18_16-22-05.jpg
Analysis plot saved to /home/kassandrarodriguez/auto-park-car/photos/analysis_plot.png
The parking spot is empty.
Finished scanning for parking spots.
camera close
Program has ended.
kassandrarodriguez@raspberrypi:~/auto-park-car $
```

Demo 2





CONCLUSION

- Overall, we developed a autonomous car that can navigate parking spots at its own through camera sightings.
- Robot car collects data from the different sensors including camera so that it can detect various obstacles amongst parking lots.
- Table parking lot was utilized with the different obstacles as well as tapes to indicate parking spots.
- Testings were executed at multiple levels to ensure working functionality of the robot.
- Utilized some design processes for both the hardware and software functionality..

References

- M. B. M. Mansour, A. Said, N. E. Ahmed and S. Sallam, "Autonomous Parallel Car Parking," 2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4), London, UK, 2020, pp. 392-397, doi: 10.1109/WorldS450073.2020.9210298.
- , M. T. Zumma, D. Halder, M. Rahman and N. Rahman, "Autonomous Path Planner Vehicle Using Raspberry PI," 2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kharagpur, India, 2022, pp. 1-7, doi: 10.1109/ICCCNT54827.2022.9984547.
- Tenhundfeld, N. L., de Visser, E. J., Ries, A. J., Finomore, V. S., & Tossell, C. C. (2020). Trust and Distrust of Automated Parking in a Tesla Model X. Human Factors, 62(2), 194-210. https://doi.org/10.1177/0018720819865412

FUTURE WORK

- Some areas of future work include Sensor Fusion algorithms where sensor data gets fused together.
- This would help to reduce the number of errors that come through when obtaining position and location information of autonomous vehicles.
- In the future, we would like to implement task scheduling into our auto parking system
    - We've researched way to achieve this and we found that using Python' s Ascyncio library could help us manage non blocking tasks for our system.
- Another feature we would like to implement in the future would be some sort of Mapping algorithm that can keep track of the robot's location within the controlled environment.

THANK YOU!!!